Lab 17 — Questions

Question 1 [Understanding]

Explain the difference between a chart, a release and a revision. Where does Helm 3/4 store a release's history, and why does that allow `helm rollback` without any server component?

Question 2 [Prediction]

```
helm install stack ./stack -n lab17 --create-namespace
helm upgrade stack ./stack --set api.replicas=3
helm upgrade stack ./stack --set api.message="v2"
helm get values stack
```

What does the last command show? How many replicas of the API run after the third command, and why? What would you change to get both three replicas and the message "v2"?

Question 3 [Diagnosis]

```
$ helm list -A
Error: kubernetes cluster unreachable: Get "http://localhost:8080/version": dial tcp
[::1]:8080: connect: connection refused
$ kubectl get nodes
NAME         STATUS   ROLES           AGE   VERSION
wsl-ubuntu   Ready    control-plane   6d    v1.34.1+k3s1
```

The cluster is running, `kubectl` sees it, `helm` does not. Explain, give the fix, and say why `kubectl` does not have the same problem.

Question 4 [Analysis]

The chart annotates the Pod template with `checksum/config: {{ include (print $.Template.BasePath "/configmap.yaml") . | sha256sum }}`. Explain what this produces on `helm upgrade --set api.message=other`, what would happen without the annotation (lab 14), and a case where this idiom triggers an unwanted redeployment.

Question 5 [Prediction]

```
helm history stack          # revisions 1, 2, 3 ; 3 = deployed
helm rollback stack 1
helm history stack
helm uninstall stack
kubectl get pvc -n lab17
```

Describe the history after the rollback (number of revisions, description of the last one), then what `kubectl get pvc` shows. Why does Helm behave this way with PVCs, and which action really deletes the data?

Question 6 [Analysis]

Compare Helm and Kustomize: model, distribution, history, use cases. A team must (a) install PostgreSQL and Prometheus provided by third parties, (b) deploy its four microservices into three

environments through Argo CD. What would you choose for each need, and why do the two often coexist?

Question 7 [Diagnosis]

```
# overlays/prod/kustomization.yaml
resources: [../../base]
images:
  - name: localhost/api
    newTag: "1.1"
labels:
  - pairs: { env: prod }
    includeSelectors: false
```

After `kubectl apply -k overlays/prod`, the API Pods do carry image 1.1 but **not** the `env=prod` label; the Deployment does. Explain, fix, and say why `includeSelectors: true` on an existing Deployment would be a bad idea.

Question 8 [Analysis]

`podman image inspect api:1.1 --format '{{.Digest}}'` prints `sha256:e1b2…`; `podman push --digestfile` writes `sha256:b820…`; `kubectl get pod -o jsonpath='{..imageID}'` shows `localhost:5000/api@sha256:b820…`. Explain why two different digests name the same image, which one to pin in production, and why a tag is not enough.

Question 9 [Diagnosis]

A Pod stays in `ImagePullBackOff` with `failed to pull image "registry.internal:5000/api:1.1": … http: server gave HTTP response to HTTPS client`. The same registry works as `localhost:5000`. Explain the difference in treatment, give the file and content to write on k3s, and what to do afterwards. What would you do on a managed cluster where you have no access to the nodes?

Question 10 [Analysis]

A pipeline runs `helm upgrade --install api ./chart --set api.image.tag=$CI_COMMIT_SHA --wait --timeout 5m` with a kubeconfig. Say what `--install`, `--wait` and `--timeout` bring, which kubeconfig the runner should have (lab 16), and what it should do if the command fails.

Question 11 [Understanding]

Explain GitOps by contrasting "CI pushes to the cluster" with "the cluster pulls from Git". What concrete advantages (audit, rollback, drift) and what consequence for what CI is still allowed to do? Relate it to the reconciliation loop of lab 11.

Question 12 [Analysis]

Your chart works on k3s with `Service: LoadBalancer`, `local-path` and the Traefik Ingress. List what will concretely change when moving to EKS/AKS/GKE, then to OpenShift, and what will not change. How do you write the chart so it stays portable?

Question 13 [Analysis]

A developer proposes: "we put `latest` everywhere, `imagePullPolicy: Always`, and CI runs `kubectl rollout restart` after every push: no more tags or charts to change". Take the proposal apart point by point (reproducibility, rollback, audit, security), then propose the correct version with the tools of this lab.